

CS396 Software Design Principles and Practices

Project: Final Report

Contributors

Evelyn Hung

Option

Option 3

Link to Project GitHub Repository

https://github.com/nu-cs-sw-design/project-20252601-396_final

Software Requirements

UC1: Start the Game

Actor: User

Precondition: The application has been launched (Main.main() has been executed), and the initial Game, Deck, Player, and GameUI objects have been instantiated.

Basic Flow:

1. GameUI prompts the user to select the number of players.
2. User selects a valid number (3-5) of players, which is set in the Game object.
3. Game gives one DEFUSE card to each player.
4. Game initializes the deck with 3 times the player count of all cards *except* the Exploding Kittens and Defuse.
5. Game shuffles the deck.
6. Game deals 4 cards to each player.
7. Game inserts (player count -1) Exploding Kittens into the deck.
8. System (via Game) shuffles the deck again.
9. System (via GameUI) announces the start of the game and prompts the first player to begin their turn.

Exception Flow A: Invalid Number of Players

- 2.a. If the user provides an invalid input (e.g., other than 3, 4, 5), the system displays an error message.
- 3.a. Resume at Step 1.

Postcondition: The game is fully initialized. The deck is populated and shuffled, players have their starting hands (including one DEFUSE card), and the game has entered the main turn loop, starting with Player 0.

UC2: Draw an Exploding Kitten

Actor: Current Player

Precondition:

1. It is the Player's turn.
2. The Player has finished playing cards (or chooses not to play any).
3. The Player must draw a card to end their turn.

Basic Flow (Player has a Defuse Card):

1. Player initiates the "draw card" action to end their turn.
2. Game draws the top card from the Deck.
3. Game identifies the drawn card is an EXPLODING_KITTEN.
4. GameUI immediately informs the Player they drew an EXPLODING_KITTEN.
5. Game checks the Player's hand and identifies they have a DEFUSE card.
6. GameUI informs the Player their DEFUSE card is being used automatically and they survived.
7. Game removes one DEFUSE card from the Player's hand.
8. GameUI prompts the Player to provide an index for where to place the EXPLODING_KITTEN back into the Deck.
9. Player provides a valid index.
10. Game re-inserts the EXPLODING_KITTEN into the Deck at the specified location.
11. GameUI confirms the card was re-inserted.
12. The Player's turn ends.

Exception Flow A (Player has no Defuse Card):

- 5.a. Game checks the Player's hand and finds no DEFUSE card.
- 6.a. GameUI informs the Player they have no DEFUSE card and have "exploded."
- 7.a. Player sets the Player's status to "dead."
- 8.a. The Player's turn ends, and they are out of the game.
- 9.a. If only 1 player is left, they are the winner. Else, pass on to the next player.

Exception Flow B (Invalid Input for Re-insertion):

- 9.a. Player provides an invalid input.
- 10.a. GameUI informs the Player the index is invalid.
- 11.a. Resume at Step 8.

Postcondition: Either the Player has been eliminated from the game, or the Player has used one DEFUSE card and returned the EXPLODING_KITTEN to the Deck. A winner is decided or turn passes to next player.

UC3: Play a Nope card

Actor: Current Player

Precondition:

1. The current player has just played an action card.
2. The action of that card (the "Original Action") has not yet resolved.
3. GameUI is in the "checking for Nopes" phase, prompting all players for a response.

Basic Flow (Single Nope / Cancellation):

1. Game initiates the Nope checking Loop, iterating through all other players.
2. Player 0 has Nope, Game prompts Player 0 to decide to Nope or not.
3. GameUI identifies that the current action is "Active" and prompts the Player: "Do you want to CANCEL the action?"
4. Player 0 chooses to play a NOPE card.
5. Game removes Nope from Player 0 's hand.
6. Game updates the PendingAction state:
 - Increments Nope count (e.g., 0 to 1).
 - Sets action status to CANCELLED (Odd count).
7. GameUI announces: "A Nope was played! The action is currently BLOCKED."
8. Game continues the loop to allow other players to respond to this new state.
9. Assume no further Nopes are played.
10. Game resolves the PendingAction:
 - The original Action Card is discarded without effect.
 - The turn returns to the original Action Player (state is reset to NORMAL).

Alternative Flow A (Reviving an Action / "Noping a Nope"): Condition: Occurs if the action is currently in a "CANCELLED" state (Step 6 of Basic Flow).

1. Game continues the loop to the next Player.
2. GameUI identifies that the current state is "Cancelled" and prompts the Player: "The action is BLOCKED. Do you want to NOPE the NOPE (Revive the action)?"
3. Player chooses to play a NOPE card.
4. Game removes the card from the Player's hand.
5. Game updates the PendingAction state:
 - Increments Nope count (e.g., 1 to 2).
 - Sets action status to ACTIVE (Even count).
6. GameUI announces: "A Nope was played! The action is REVIVED and will proceed."
7. Assume no further Nopes are played.
8. Game resolves the PendingAction: The original Action Card executes its logic.

Alternative Flow B (Player Declines/No Card):

1. Game checks the Player.
2. Game identifies the Player is either "Dead" or has no NOPE cards.

3. Game automatically skips the Player.
4. OR (if Player has card but declines): Player selects "No".
5. Game proceeds to the next player in the loop.

Postcondition: The NOPE_PHASE ends. The Original Action has either executed (Even Nopes) or been discarded (Odd Nopes). All played **NOPE** cards are discarded.

UC4: Play a Shuffle Card

Actor: Current Player

Precondition:

1. It is the Player's turn.
2. The Player has one or more SHUFFLE cards in their hand.
3. The Player has not yet drawn a card to end their turn.

Basic Flow (Action Succeeds):

1. Player chooses to play the SHUFFLE card from their hand.
2. GameUI processes the card selection.
3. Game removes the SHUFFLE card from the Player's hand.
4. GameUI asks all other players if they wish to play a NOPE card.
5. No other players play a NOPE card (or an even number of NOPE cards are played).
6. Game triggers the deck shuffling.
7. The Deck.shuffle() method randomizes the order of all cards currently in the deck.
8. Game announces to all players that the deck has been shuffled.
9. The Player's turn continues (they may play more cards or choose to draw).

Alternate Flow A (Action is Noped):

- 4.a. GameUI asks all other players if they wish to play a NOPE card.
- 5.a. Another player plays a NOPE card.
- 6.a. An odd number of NOPE cards are played in total, and the SHUFFLE action is canceled.
- 7.a. The SHUFFLE card remains in the discard pile, and the shuffle method is not called.
- 8.a. GameUI announces that the action was Noped.
- 9.a. The Player's turn continues.

Postcondition: The SHUFFLE card is discarded. If the action was successful, the deck is now in a new, random order. If the action was Noped, the deck's order is unchanged. The Player's turn continues.

UC5: Play a Swap Top And Bottom Card

Actor: Current Player

Precondition:

1. It is the Player's turn.
2. The Player has one or more SWAP_TOP_AND_BOTTOM cards in their hand.
3. The Player has not yet drawn a card to end their turn.

Basic Flow (Action Succeeds):

1. Player chooses to play the SWAP_TOP_AND_BOTTOM card from their hand.
2. GameUI processes the card selection.
3. Game removes the SWAP_TOP_AND_BOTTOM card from the Player's hand.
4. GameUI asks all other players if they wish to play a NOPE card.
5. No other players play a NOPE card (or an even number of NOPE cards are played).
6. Game executes the card's effect.
7. The Game object (via the Deck) draws the top card and the bottom card, then re-inserts the original top card at the bottom and the original bottom card at the top.
8. GameUI announces to all players that the top and bottom cards of the deck have been swapped.
9. The Player's turn continues (they may play more cards or choose to draw).

Alternate Flow A (Action is Noped):

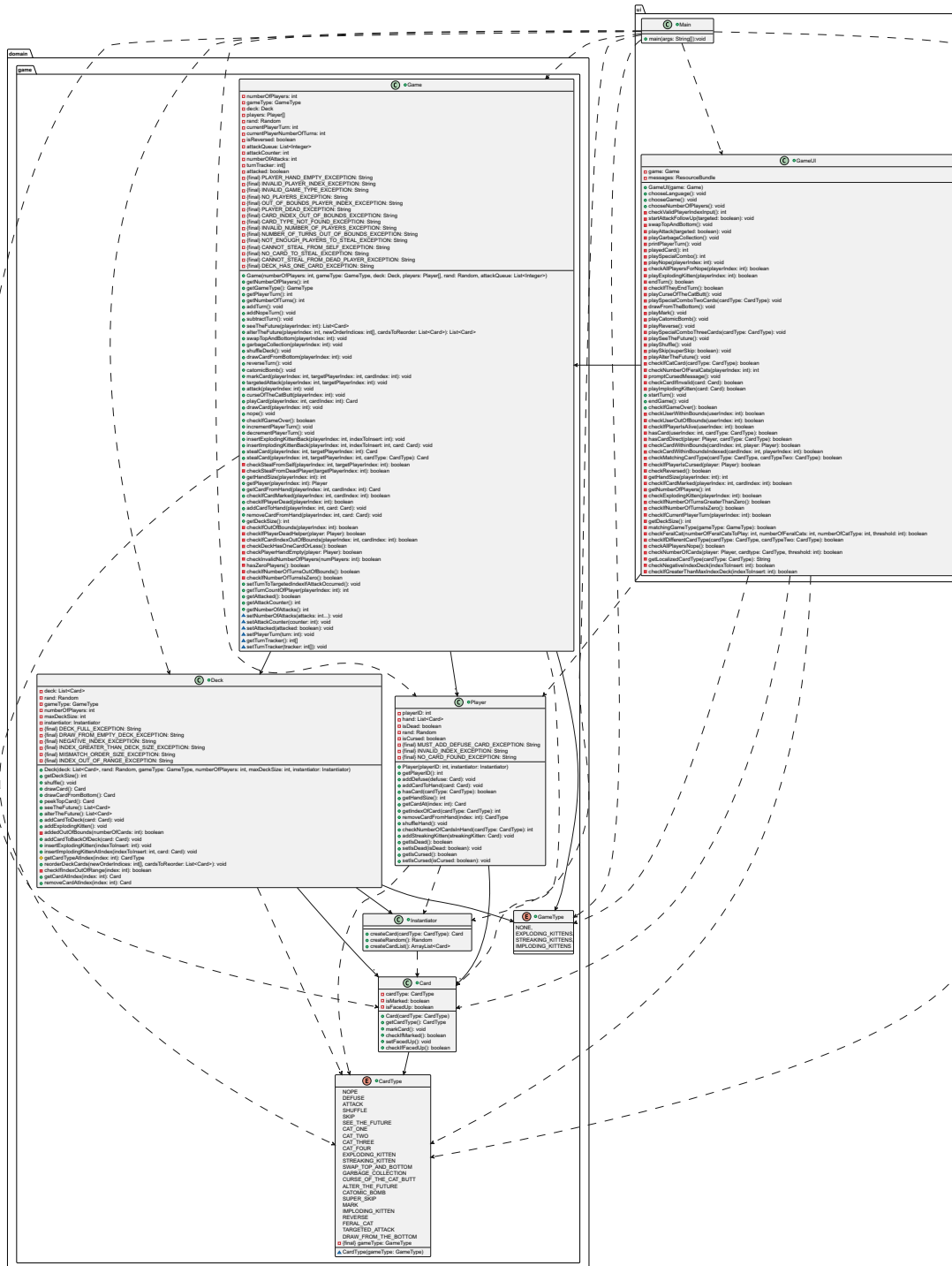
- 4.a. GameUI asks all other players if they wish to play a NOPE card.
- 5.a. Another player plays a NOPE card.
- 6.a. An odd number of NOPE cards are played in total, and the action is canceled.
- 7.a. The swap top and bottom method is not called.
- 8.a. GameUI announces that the action was Noped.
- 9.a. The Player's turn continues.

Exception Flow A (Not Enough Cards in Deck):

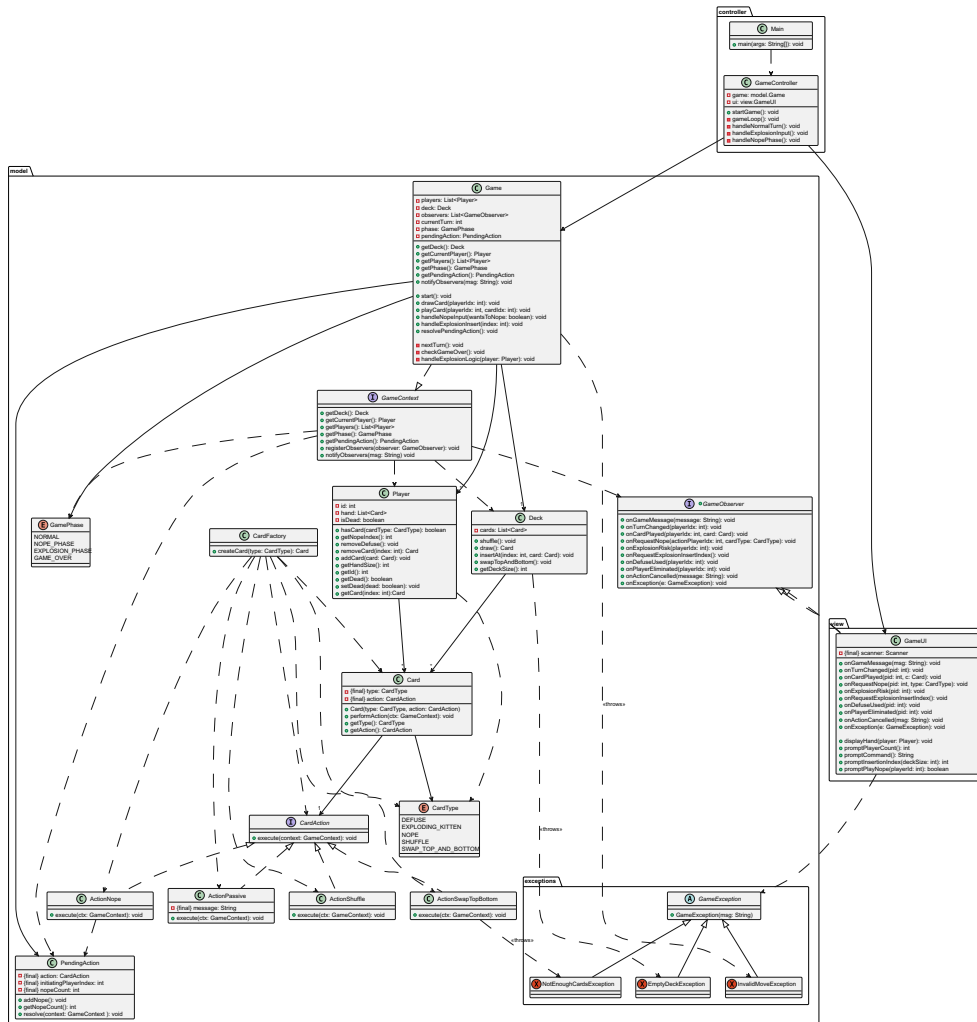
- 6.a. Deck checks the deck size and finds it is one or less.
- 7.a. The action fails and throws a NotEnoughCardsException.
- 8.a. GameUI informs the player the action failed as there are not enough cards.
- 9.a. The Player's turn continues.

Postcondition: The SWAP_TOP_AND_BOTTOM card is discarded. If the action was successful and valid, the top and bottom cards of the Deck have been exchanged. The Player's turn continues.

Initial Design Diagram



Design Diagram



Analysis of the Changes

1. Architectural change: One god class (Game) to MVC pattern
 - Change:
 - Before: The Game class was a "God Object." It managed game rules, player turns, deck logic, and contained exception strings.
 - After: The system is split into Model (Game, Deck, Player), View (GameUI), and Controller(GameController).
 - Analysis: The original design suffered from Low Cohesion. The Game class knew too much and did too much. By separating responsibilities to smaller classes, the code is now easier to test and maintain. The GameController now handles the flow, Game only manages state, and GameUI only handles I/O.
 - Principles Applied:

- 10. Single Responsibility Principle: Each class now has one reason to change (UI changes affect only GameUI, Rules affect only Game).
 - 15. High Cohesion: Related functions are grouped into specific layers.
- 2. Applied Strategy Pattern for Card Actions
 - Change:
 - Before: The Game class had specific methods for every card type.
 - After: The Game class delegates the method “execute” to a CardAction interface. Concrete strategies like ActionPassive, ActionShuffle, ActionNope, and ActionSwapTopBottom implement the specific logic.
 - Analysis: The old design violated the Open/Closed Principle. Adding a new card type required modifying the Game class. The new design allows adding new cards by simply creating a new class that implements CardAction, without touching existing code.
 - Principles Applied:
 - 3. Information Hiding: The Game doesn't know how the action works.
 - 4. Encapsulate what varies: Card behaviors vary, so they are encapsulated in their own classes.
 - 6. Program to interface, not implementation: The Card class depends on the CardAction interface, not specific implementation details.
 - 5. Favor composition over inheritance: Card is composed of a behavior (action), making it flexible.
- 3. Applied Observer Pattern
 - Change:
 - Before: The GameUI was tightly coupled to specific methods in Game.
 - After: The Game class implements a notification system (notifyObservers), and GameUI implements the GameObserver interface to react to events.
 - Analysis: This decouples the Model from the View. The Game class no longer needs to know how the game is displayed. It just broadcasts events.
 - Principles Applied:
 - 7. Strive for loosely coupled designs: The Model knows it has observers, but doesn't know who they are.
 - 14. Dependency Inversion Principle: The high-level module (Game) does not depend on the low-level module (GameUI). Both depend on an abstraction (GameObserver).
- 4. Applied custom exceptions for error handling
 - Change:
 - Before: Errors were handled via a long list of String constants in the Game class (e.g., PLAYER_HAND_EMPTY_EXCEPTION).
 - After: Add an exceptions package with specific classes (EmptyDeckException, InvalidMoveException) inheriting from a base GameException.
 - Analysis: Using Strings for errors is prone to typos. Custom exceptions provide type safety and allow the Controller to handle specific error scenarios more gracefully.

- Principles Applied:
 - 1. Encapsulation: Error details are encapsulated within exception objects rather than exposed as public constants.

5. Introduction of Explicit State Management (GamePhase)

- Change:
 - Before: Game state was likely implicit or managed by flags and variables scattered mainly in GameUI and Game.
 - After: An explicit GamePhase enum (NORMAL, NOPE_PHASE, EXPLOSION_PHASE) dictates what actions are legal.
- Analysis: This acts as a State Machine. It prevents illegal moves (like playing a Shuffle card while resolving an Explosion) by explicitly checking the phase. It makes the control flow predictable.
- Principles Applied:
 - 15. High Cohesion: Instead of state logic being scattered across multiple variables, it is unified into a single concept (GamePhase).
 - 1. Encapsulation: The "State" of the application is encapsulated into a specific field, protecting the game flow from being put into an invalid state manually.

6. Introduction of PendingAction for Complex Interactions

- Change:
 - Before: The logic for "Noping" was implemented using recursion mixed directly with Blocking UI (Scanner).
 - After: A PendingAction class stores the target action and an integer nopeCount. The resolution logic is determined by checking the state (Odd vs Even) stored in this object.
- Analysis: The original recursive approach was brittle and hard to unit test because it halted code execution to wait for console input inside the logic loop. By reifying the concept of an interrupted action, we separated the User Interface flow (looping through players) from the Game Logic (calculating if the action is cancelled).
- Principles Applied:
 - 1. Encapsulation: We encapsulated the state of the "interrupt" (the nopeCount and the original action) into a dedicated object (PendingAction), rather than relying on the transient state of recursive method calls.
 - 7. Strive for loosely coupled designs: The Model no longer relies on blocking UI calls to determine the outcome of a card.

7. Refactoring the Deck Class

- Change:
 - Before: Deck contained references to GameType, Instantiator, and game-specific rules (removeBombs).

- After: Deck is a simple data structure managing a List<Card> with standard operations (draw, shuffle, insertAt).
- Analysis: The original Deck had "Feature Envy"—it was trying to manage game rules instead of just managing cards. The refactored Deck is highly cohesive and reusable.
- Principles Applied:
 - 15. High Cohesion: The class now focuses on one thing: managing the collection of cards.
 - 9. Principle of Least Knowledge: The Deck no longer needs to know about GameType or Instantiator.